Lab 06 — Writing Your Own Rules (the ModSecurity Rule Language)

> **Level:** Intermediate → Advanced **Goal:** Learn the `SecRule` language and write, load and test **your own** rules — the day-to-day work of a WAF engineer. **Time:** ~50 minutes **Operator focus:** CRS covers generic attacks. Every real deployment also needs *app-specific* rules — block an internal path, enforce a header, add a virtual patch. That is **your** code, and this lab is how you write it.

---

1. Theory — anatomy of a `SecRule`

Every ModSecurity rule follows the same shape:

```
SecRule VARIABLES "OPERATOR argument" "ACTIONS"
```

```
SecRule  REQUEST_URI  "@beginsWith /admin"  "id:1000001,phase:1,deny,status:403,log,msg:'bloc
         └─────┬─────┘  └────────┬────────┘  └──────────────────────┬──────────────────────
          WHERE to look     WHAT to match                  WHAT to do
```

The four things you always specify

Part	Question it answers	Examples
Variables	*Where* in the request do I look?	`REQUEST_URI`, `ARGS`, `ARGS:data`, `REQUEST_HEADERS:User-Agent`, `REQUEST_METHOD`, `REQUEST_BODY`
Operator	*What* pattern am I matching?	`@rx` (regex), `@beginsWith`, `@streq`, `@contains`, `@ipMatch`, `@detectSQLi`
Transformations (`t:`)	How do I *normalise* input first, to defeat evasion?	`t:lowercase`, `t:urlDecodeUni`, `t:removeNulls`, `t:none`
Actions	*What do I do* on a match?	`deny`, `pass`, `block`, `status:403`, `log`, `msg:'…'`, `setvar:…`, `chain`, `id:…`, `phase:…`

The two things every rule *must* have

- `id:` — a unique numeric ID. **Local/custom rules must use the `1000000`–`1999999` range**, which is reserved so your rules never collide with CRS's IDs.
- `phase:` — *when* the rule runs during request processing.

Processing phases (this matters — see the bug we hit below)

Phase	Fires when	You inspect
1	Request headers received	URI, headers, method
2	Request body received	POST args, body, everything in phase 1
3	Response headers ready	response status/headers
4	Response body ready	response body (data-leak detection)
5	Logging	—

> **Why phase matters:** CRS *initialises* the anomaly score in phase 1 (rule `901xxx`) and *evaluates the block* in phase 2 (rule `949110`). If you write a scoring rule in phase 1 that loads **before** initialization, the init resets your score to zero. We hit exactly this bug in Step 4 — it's a classic beginner mistake.

---

2. Practical — write a custom rules file

Create `rules/LOCAL-999-CUSTOM.conf` in the repo (already provided). It holds four rules that show the common patterns:

```
# Rule 1: hard block any access to /admin.
SecRule REQUEST_URI "@beginsWith /admin" \
    "id:1000001,phase:1,deny,status:403,log,\
     msg:'ModSecLabs: access to /admin blocked',tag:'local-policy'"

# Rule 2: block a forbidden header value.
SecRule REQUEST_HEADERS:X-Debug "@streq enable" \
    "id:1000002,phase:1,deny,status:403,log,\
     msg:'ModSecLabs: X-Debug header not allowed'"

# Rule 3: CHAINED rule -- block POST /api/log only when data contains 'secret'.
SecRule REQUEST_METHOD "@streq POST" \
    "id:1000003,phase:2,deny,status:403,log,\
     msg:'ModSecLabs: secret leak attempt',chain"
    SecRule ARGS:data "@rx (?i)secret" "t:none,t:lowercase"

# Rule 4: contribute to the anomaly SCORE instead of hard-blocking.
SecRule REQUEST_HEADERS:User-Agent "@rx (?i)evil-scanner" \
    "id:1000004,phase:2,pass,log,\
     msg:'ModSecLabs: suspicious scanner UA (scored)',\
     setvar:'tx.inbound_anomaly_score_pl1=+5'"
```

What each rule teaches

- **Rule 1** — the simplest possible rule: a variable, an operator, `deny`.
- **Rule 2** — inspecting a specific header by name (`REQUEST_HEADERS:X-Debug`).
- **Rule 3** — a **chained** rule: `chain` links two `SecRule` lines so the action fires **only if both** match. This is how you avoid over-blocking (block `POST` *and* suspicious data, not every `POST`).

- **Rule 4** — the CRS-native pattern: instead of `deny`, use `pass` + `setvar` to **add to the anomaly score**. The request is blocked by `949110` only if the *total* crosses the threshold. This lets your rule cooperate with CRS scoring instead of fighting it.

---

3. Load your rules into the WAF

Custom rule files are mounted into the CRS rules directory:

```
docker run -d --name modseclabs-waf-custom \
  --add-host=host.docker.internal:host-gateway \
  -e BACKEND="http://host.docker.internal:5000" -e PORT=8080 \
  -e MODSEC_RULE_ENGINE=On -e PARANOIA=1 -e ANOMALY_INBOUND=5 \
  -e MODSEC_AUDIT_LOG=/dev/stdout -e MODSEC_AUDIT_ENGINE=RelevantOnly \
  -v "$PWD/rules/LOCAL-999-CUSTOM.conf:/etc/modsecurity.d/owasp-crs/rules/LOCAL-999-CUSTOM.co
  -p 8091:8080 owasp/modsecurity-crs:nginx
```

Always check the config loaded cleanly (a syntax error stops nginx):

```
docker logs modseclabs-waf-custom | grep -iE "error|emerg" | head
```

No output = good.

---

4. Test every rule

```
curl -s -o /dev/null -w "R1 /admin            -> %{http_code}\n"  "http://localhost:8091/admi
curl -s -o /dev/null -w "R2 X-Debug: enable    -> %{http_code}\n" -H "X-Debug: enable" "http:
curl -s -o /dev/null -w "R3 POST w/ secret     -> %{http_code}\n" --data "data=my secret toke
curl -s -o /dev/null -w "R3 POST harmless      -> %{http_code}\n" --data "data=hello world"
curl -s -o /dev/null -w "R4 evil-scanner UA    -> %{http_code}\n" -H "User-Agent: evil-scanne
curl -s -o /dev/null -w "Clean request         -> %{http_code}\n" "http://localhost:8091/"
```

Expected:

```
R1 /admin            -> 403
R2 X-Debug: enable    -> 403
R3 POST w/ secret     -> 403
R3 POST harmless      -> 200      <- chain's second condition failed, so no block
R4 evil-scanner UA    -> 403      <- scored +5, crossed threshold 5
Clean request         -> 200
```

Confirm your rules fired by name in the audit log:

```
docker logs modseclabs-waf-custom | grep -o "ModSecLabs:[^\"']*" | sort -u
# ModSecLabs: X-Debug header not allowed
# ModSecLabs: access to /admin blocked
```

```
# ModSecLabs: secret leak attempt
# ModSecLabs: suspicious scanner UA (scored)
```

---

5. The phase bug you WILL hit (a real war story)

When Rule 4 was first written as `phase:1`, it did **not** block — the scanner UA returned `200` even though the score should have crossed the threshold. Why?

- CRS files are loaded **alphabetically**. `LOCAL-999-CUSTOM.conf` sorts **before** `REQUEST-901-INITIALIZATION.conf` (`L` < `R`).
- In phase 1, our rule ran **first** and added `+5` … then `901` initialisation ran and **reset the score to 0**.

Moving the rule to `phase:2` fixed it: initialization already happened in phase 1, so by phase 2 our `+5` sticks and `949110` (also phase 2) sees it.

> **The lesson every WAF engineer learns:** *rule ordering and phase are as important as the match itself.* If a rule "isn't working," 90% of the time it's the wrong phase, the wrong load order, or a missing transformation — not the pattern. Always test the actual behaviour; never assume a rule works because it "looks right."

---

6. Transformations — defeating evasion

Attackers encode payloads to slip past naive matches. Transformations normalise input **before** the operator runs. Compare:

```
# Fragile: misses URL-encoded or mixed-case input
SecRule ARGS:data "@rx secret" "id:1000010,phase:2,deny"

# Robust: decode and lowercase first
SecRule ARGS:data "@rx secret" \
    "id:1000011,phase:2,deny,t:none,t:urlDecodeUni,t:lowercase,t:removeNulls"
```

The second one catches `SeCrEt`, `%73ecret`, and `sec%00ret`. **Every serious rule you write should think about what transformations an attacker's evasion would require** — this is the single biggest quality difference between a beginner's rule and a professional one.

Clean up

```
docker rm -f modseclabs-waf-custom
```

---

7. Recap

- `SecRule VARIABLES "OPERATOR arg" "ACTIONS"` — where, what, do-what.
- Custom IDs live in `1000000–1999999`; every rule needs `id:` and `phase:`.
- **Phase and load order** decide whether your rule even runs at the right time — the #1 source of "my rule doesn't work" bugs.
- Use `chain` to combine conditions and avoid over-blocking.
- Prefer `setvar` scoring over hard `deny` so your rules cooperate with CRS.
- Always apply **transformations** to defeat encoding-based evasion, and always **test the real behaviour**.

Next: Lab 07 — the other half of the operator's job: hunting down and fixing **false positives** without punching holes in your protection.